SIEME Communication Driver

Driver for Serial Communication
between Studio and Siemens Devices
Using MPI Interfaces

Contents

INTRODUCTION **2**

GENERAL INFORMATION **3**

DEVICE CHARACTERISTICS 3
LINK CHARACTERISTICS 3
DRIVER CHARACTERISTICS 3
CONFORMANCE TESTING 4

INSTALLING THE DRIVER **5**

CONFIGURING THE DRIVER **6**

SETTING THE COMMUNICATION PARAMETERS 6
CONFIGURING THE DRIVER WORKSHEET 7
MAIN DRIVER SHEET (MDS) 14

EXECUTING THE DRIVER **16**

TROUBLESHOOTING **17**

SAMPLE APPLICATION **22**

REVISION HISTORY **23**

Introduction

The SIEME driver enables communication between the Studio system and the Siemens devices using MPI interfaces, according to the specifications discussed in this publication.

This publication was designed to help you install, configure and execute the SIEME driver to enable communication with the Siemens devices. The information in this publication is organized as follows:

- **Introduction**: Provides an overview of the SIEME driver documentation
- **General Information**: Provides information needed to identify all the required components (hardware and software) used to implement communication between Studio and the SIEME driver
- **Installing the Driver**: Explains how to install the SIEME driver
- **Configuring the Driver**: Explains how to configure the communication driver
- **Executing the Driver**: Explains how to execute the driver to verify that you installed and configured the driver correctly
- **Troubleshooting**: Lists the most common error codes for this protocol and explains how to fix these errors
- **Sample Application**: Explains how to use a sample application to test the driver configuration
- **Revision History**: Provides a log of all modifications made to the driver and the documentation

Notes:

- This document assumes that you have read the “Development Environment” chapter in the product’s *Technical Reference Manual*.
- This document also assumes that you are familiar with the Windows XP/7/8 environment. If you are unfamiliar with Windows XP/7/8, we suggest using the **Help** feature (available from the Windows desktop **Start** menu) as you work through this guide.

General Information

This chapter explains how to identify all the hardware and software components used to implement communication between the SIEME driver and Siemens devices using MPI interfaces.

The information is organized into the following sections:

- Device Characteristics
- Link Characteristics
- Driver Characteristics
- Conformance Testing

Device Characteristics

This driver has been tested successfully with the following devices:

- **Manufacturer**: Siemens
- **Compatible Equipment**: Any Siemens PLC that is compatible with the MPI protocol
- **Siemens PLC Programmer Software**: Step 7

Link Characteristics

To establish communication, you must use links with the following specifications:

- **Device Communication Port**: Serial Port
- **Physical Protocol**: RS232/RS485
- **Logic Protocol**: MPI Protocol
- **Adapters/Converters**: PC Adapter (MPI)

Driver Characteristics

The SIEME driver is composed of the following files:

- **`SIEME.INI`**: Internal driver file. *You must not modify this file*.
- **`SIEME.MSG`**: Internal driver file containing error messages for each error code. *You must not modify this file*.
- **`SIEME.PDF`**: Document providing detailed information about the SIEME driver
- **`SIEME.DLL`**: Compiled driver

> **Notes:**
> - All of the preceding files are installed in the **`/DRV`** subdirectory of the Studio installation directory.
> - The SIEME driver requires the **`AGLINK.DLL`** (version 1.03 or higher), which is installed with the product

You can use the SIEME driver on the following operating systems:

- Windows XP/7/8/8.1/2008/2012
- Windows Embedded Compact (Windows CE 5/6/7), Windows Embedded Standard

The SIEME driver supports the following registers:

Register Type	Write	Read	Bit	Byte	Word	DWord	Float	String
M (*Flags*)	•	•	•	•	•	•	•	•
T (*Timers*)	•	•	–	–	•	–	–	•
Z or C (*Counters*)	•	•	–	–	•	–	–	•
E or I (*Inputs*)	–	•	•	•	•	•	•	•
A or Q (*Outputs*)	•	•	•	•	•	•	•	•
DB (*Data Blocks*)	•	•	•	•	•	•	•	•

Conformance Testing

The following hardware/software was used for conformance testing:

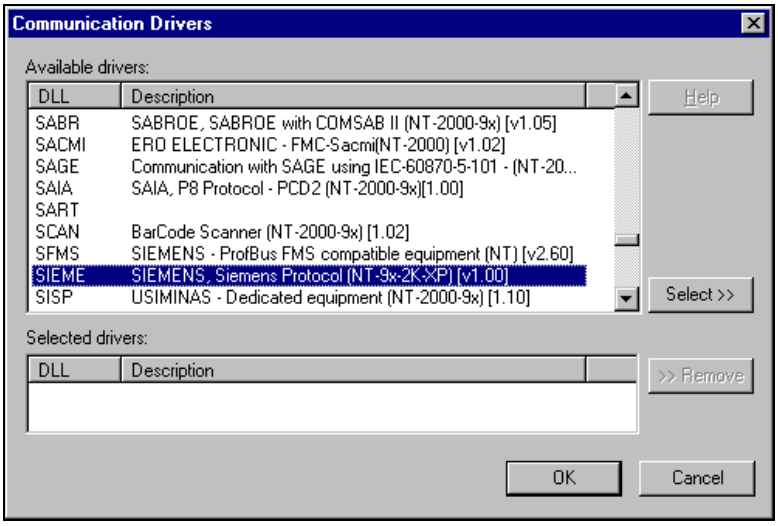
- **Equipment**: SIMATIC S7-315 2DP
- Adapters/Converters: PC Adapter MPI (Systeme Helmholz SSW7 700-751-1VK11 and Siemens 6ES7 972-0CA 23-0XA0)
- **Driver Configuration**:
  - **PLC Program**: Step 7
  - **Protocol**: MPI Protocol
  - **Communication**: Asynchronous Half-Duplex
  - **COM Port**: COM1
- **Operating System**: Windows 7, WinCE 5
- **Studio Version**: 7.1 + SP2
- **Driver Version**: 10.8

Driver Version	Studio Version	Operating System	Equipment
10.8	7.1 + SP2	Windows 7	SIMATIC S7-315 2DP

Installing the Driver

Perform the following steps to select the driver from within the application:

1. Open Studio from the **Start** menu.
2. From the Studio main menu bar, select **File →Open Project** to open your application.
3. Select **Insert →Driver** from the main menu bar to open the *Communication Drivers* dialog.
4. Select the **SIEME** driver from the *Available Drivers* list, and then click the **Select** button:



Communication Drivers Dialog

5. When the **SIEME** driver displays in the *Selected Drivers* list, click the **OK** button to close the dialog.

> ➲ **Caution:**
> For safety reasons, you must use caution when installing the physical hardware. Consult the hardware manufacturer's documentation for specific installation instructions.

Configuring the Driver

After opening Studio and selecting the SIEME driver, you must configure the driver. Configuring the SIEME driver is done in two parts:

- Specifying communication parameters
- Defining communication tags and controls in the Communication tables or *Driver* worksheet

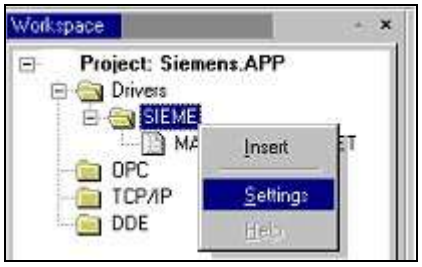
  Worksheets are divided into two sections, a *Header* and a *Body*. The fields contained in these two sections are standard for all communications drivers — except the **Station**, **Header** and **Address** fields, which are driver-specific. This document explains how to configure the **Station**, **Header** and **Address** fields only.

> **Notes:**
> For a detailed description of the Studio *Standard* and *MAIN* Driver Worksheets, and information about configuring the standard fields, review the product's *Technical Reference Manual*.

Setting the Communication Parameters

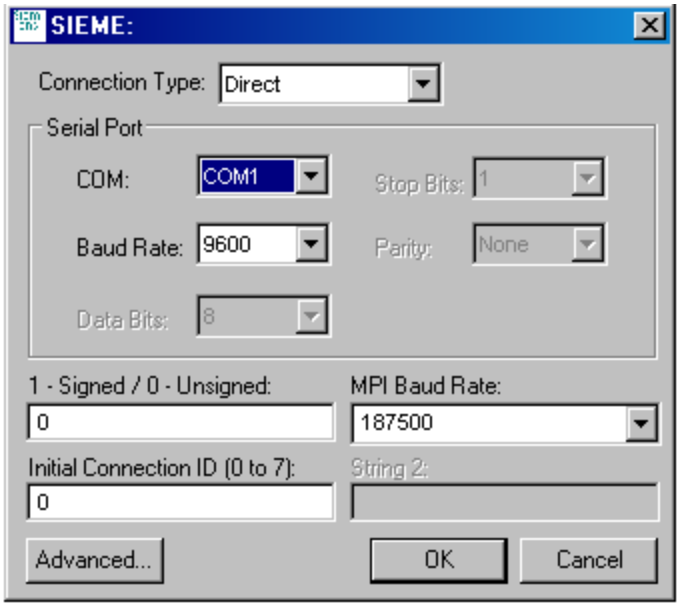
Use the following steps to configure the communication parameters, which are valid for all driver worksheets configured in the system:

1. From the Studio development environment, select the **Comm** tab located below the *Workspace*.
2. Click on the *Drivers* folder in the *Workspace* to expand the folder.
3. Right-click on the *SIEME* subfolder. When the pop-up menu displays, select the **Settings** option:



Select Settings from the Pop-Up Menu

The *SIEME: Communication Parameters* dialog displays:



SIEME: Communication Parameters Dialog

Parameters	Default Values	Valid Values	Description
1-Signed 0-Unsigned Value	0	0 or 1	This value will be default if you do not specify the Unsigned or Signed parameter for each format address. 0: Unsigned values 1: Signed values
MPI Baud Rate	19200 187500 1500000	187500	Baud Rate of MPI Bus
Initial ID Connection	0	0 to 7	This value will be the initial ID Connection. This field must be configured only if you are going to use more than one instance of this driver. For each different instance of this driver, use different IDs.

4. Verify the **COM** and **Baud Rate** settings, and change them if necessary.
5. Click **OK** to close the dialog.

Configuring the Driver Worksheet

This section explains how to configure a *Standard Driver Worksheet* (or Communication table) to associate application tags with the PLC addresses. You can configure multiple *Driver* worksheets — each of which is divided into a *Header* section and *Body* section.

Use the following steps to create a new *Standard Driver* worksheet:

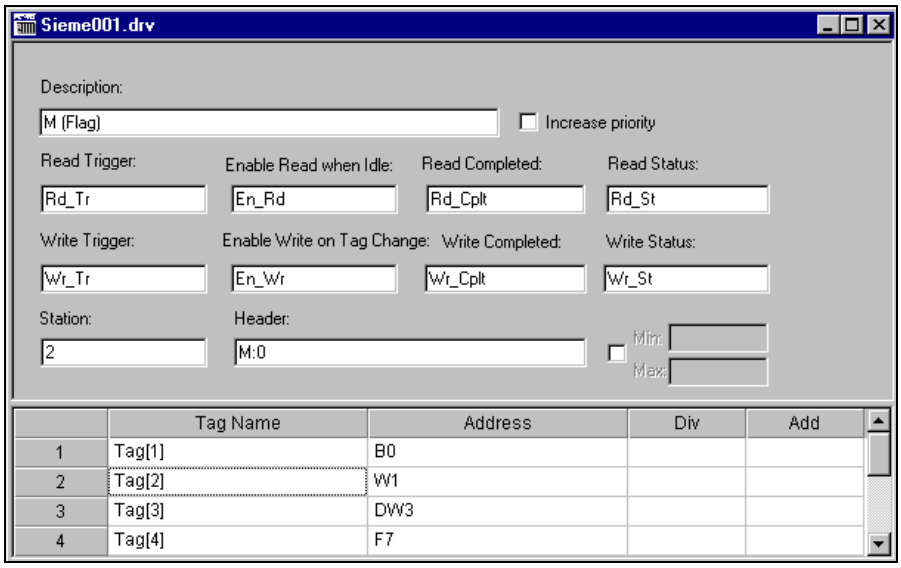
1. From the Studio development environment, select the **Comm** tab, located below the *Workspace* pane.
2. In the *Workspace* pane, expand the *Drivers* folder and right-click the *SIEME* subfolder.
3. When the pop-up menu displays, select the **Insert** option:



Inserting a New Worksheet

> **Note:**
> To optimize communication and ensure better system performance, you must tie the tags in different driver worksheets to the events that trigger communication between each tag group and the period in which each tag group must be read or written. Also, we recommend configuring the communication addresses in sequential blocks to improve performance.

The *SIEME.drv* dialog displays (similar to the following figure):



SIEME Driver Worksheet

In general, all parameters on the *Driver* worksheet (except the **Station**, **Header** and **Address** fields) are standard for all communication drivers, and they will not be discussed in this publication. For detailed information about configuring the standard parameters, consult the Studio *Technical Reference Manual*.

4. Use the following information to complete the **Station**, **Header**, and **Address** fields on this worksheet:
   - **Station** field: Use this field to specify the PLC address (*PLC ID*). Valid values are 1– 255.
   - **Header** field: Use the information in the following table to define the type of variables that will be read from or written to the device and a reference to the initial address.

     These variables must comply with the following syntax:
     - For Flags, Timers, Counters, Inputs and Outputs:

       `<Type>:<AddressReference>` (for example: **`M:1`**)
     - For Data-Blocks:

<*Type*><*TypeGroup*>:<*AddressReference*> (for example: **DB2:0**) or

<*Type*><*TypeGroup*>.DB :<*AddressReference*> (for example: **DB2.DB, DB2.DB:0**)

Where:

- **Type** is the register type. (**M**=Flags, **T**=Timers, **Z** or **C**=Counters, **E** or I=Inputs, **A** or **Q**=Outputs, and **DB**=Data Blocks)
- **TypeGroup** is the group number of the configured register type (for Data-Block types only).
- **AddressReference** (optional) is the initial address reference of the configured group. This number *always* refers to the *Byte address number*. (See the following table.)

The following table lists all of the valid initial address (reference) values for the SIEME driver:

Header Address	Siemens Address		
Byte Address Number	Byte Address Number	Word Address Number	
Byte 0	Byte 0	W0	
Byte 1	Byte 1		W1
Byte 2	Byte 2	W2	
Byte 3	Byte 3		W3
Byte 4	Byte 4	W4	
Byte 5	Byte 5		W5
Byte 6	Byte 6	W6	
Byte 7	Byte 7		W7
Byte 8	Byte 8	W8	
Byte 9	Byte 9		W9
Byte 10	Byte 10	W10	
Byte 11	Byte 11		

The next table lists all of the data types and address ranges that are valid for the SIEME driver:

Header Field Information			
Data Types	Sample Syntax	Valid Range of Initial Addresses	Comments
Flags	**M:1** or **M**	Varies according to the equipment	Logical Flags
Timers	**T:2** or **T**	Varies according to the equipment	Timer Values
Counters	**Z:10**, **C:10 or Z,C**	Varies according to the equipment	Counter Values
Inputs	**E:5**, **I:5** or **E,I**	Varies according to the equipment	Physical Input Values
Outputs	**A:8**, **Q:8** or **A,Q**	Varies according to the equipment	Physical Output Values
Data Blocks	**DB2:1,** or **DB2.DB** or **DB2.DB:1**	Varies according to the equipment	Data block values, where: ▪ Number following DB(2) specifies the data block number ▪ Number following the colon specifies the word offset in the data block

- **Address** field: Use the information provided in the following table to associate each tag to its respective device address.

  Type the tag from your application database into the **Tag Name** column. This tag will receive values from or send values to an address on the device. The address must comply with the following syntax:

  [Signed/Unsigned]`<Format><AddressOffset>.<Bit>` (for example: `X10.2`)

  `<Format><AddressOffset>.<Len>` (for example: `ST2.10, SST2.10`) – String format only

  Where:

  - **[*Signed / Unsigned*]** (*optional parameter used for integer values only*): If you do not specify this parameter, Studio inserts an integer value based on the parameters you set in the *Communication Parameters* dialog. Valid values are **S** (signed) and **U** (unsigned). Dword does not use **U** (unsigned).
  - **Format** defines how Studio treats the value read from or written to the device. (**X=**Bit, **B**=Byte or Bit, **W**=Word, **D** or **DW**=Dword, **F**=Float, **ST**=String, , **SST**= S7 String format).
    - For Data-Blocks:

      T=Timer Format (S5T), **TS**=Timer in String format (hh:mm:ss:ms), **C**=Counter Format
  - **AddressOffset** is a parameter added to the **AddressReference** parameter (configured in the **Header** field) to compose the group address configured in the **Header** field.
  - **Bit** is the bit number (from 0 – 7) from the **Byte** address. This parameter is optional, and it is supported only when the format is Byte or Bit (**B** or **X**).
  - `<Len>` is the length to read or to write. It is in bytes. String format only.

> **Note:**
> T, TS and C data formats are most used for Data-Blocks operand, however they can be used for all operands.

Sample Address Configuration		
Address on the Device	**Header Field**	**Address Field**
M (Word 5 = Byte 5 / Byte 6 / String 7, Length 10)	M	W5
	M:5	W0
	M:3	W2
M (Byte 5)	M	B5
	M:5	B0
	M:1	B4
M (Byte 6)	M	B6
	M:6	B0
	M:3	B3
M (String 7, Length 10)	M	ST7.10
	M:7	ST0.10
	M:4	ST3.10
T (33)	T	33
	T:30	3
	T33	0

Sample Address Configuration		
Address on the Device	**Header Field**	**Address Field**
T(35)	T	35
	T:35	0
	T31	4
C (3)	C	3
	C:3	0
	C:2	1
C (4)	C	4
	C:4	0
	C:2	2
DB5 (Word 2 = Byte 2 / Byte 3 / String 4 length 10)	DB5:0	W2
	DB5:2	W0
	DB5:1	W1
DB5 (Byte 2)	DB5.DB	B2
	DB5:2	B0
	DB5:1	B1
DB5 (Byte 3)	DB5.DB:0	B3
	DB5:3	B0
	DB5:2	B1
DB5 (String 4 length 10)	DB5:0	ST4.10
	DB5:4	ST0.10
	DB5:2	ST2.10
DB5 (Word 7 = Byte 7 / Byte 8 / Double Word 3)	DB5:0	W7
	DB5:7	W0
	DB5:4	W3
DB5 (Byte 7)	DB5:0	B7
	DB5:7	B0
	DB5:4	B3
DB5 (Byte 8)	DB5:0	B8
	DB5:8	B0
	DB5:4	B4
DB5 (Double Word 3)	DB5.DB	D3
	DB5:0	DW3
	DB5:DB:2	DW1
Input (Address 1, bit 4)	I:0 or E:0	X1.4
	I:1 or E:1	X0.4
	I:0 or E:0	B1.4
Input (Byte 1)	I:0 or E:0	B1
	I:1 or E:1	B0
Output(Address 1, bit 4)	Q:0 or A:0	X1.4
	Q:1 or A:1	X0.4
	Q:0 or A:0	B1.4
Output(Byte 1)	Q:0 or A:0	B1
	Q:1 or A:1	B0

➲ **Caution:**

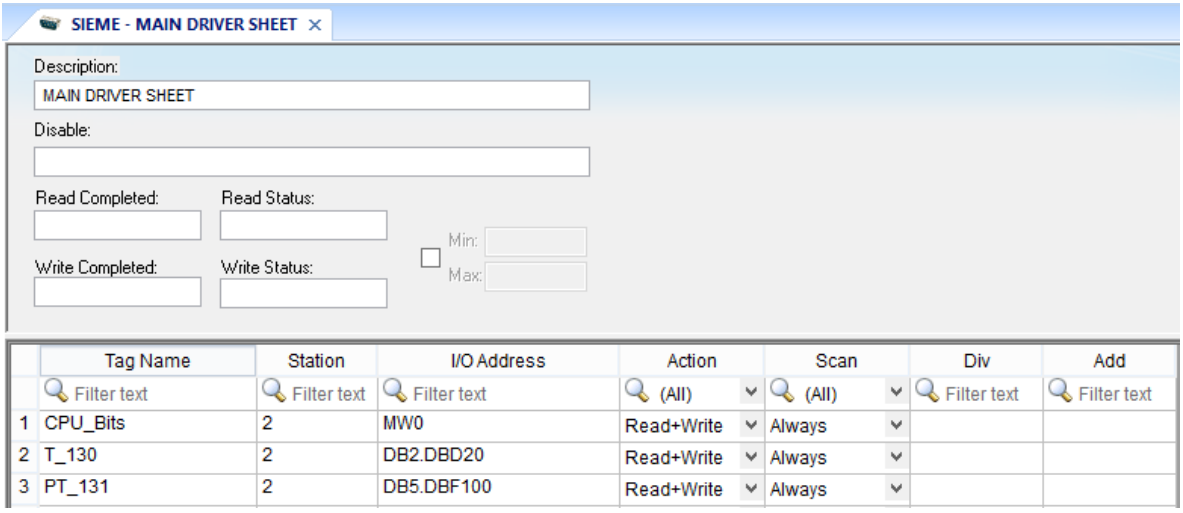
You must not configure a range of addresses greater than the maximum block size (data buffer length) supported by the protocol within the same worksheet. The maximum data buffer length for this driver is 1024 bytes per *Standard Driver* worksheet.

Main Driver Sheet (MDS)

When the driver is inserted in the application, the *MAIN DRIVER SHEET* is automatically added to the driver folder.



The MAIN DRIVER SHEET provides a simple way to associate Studio tags to addresses in the PLC. Most of the MAIN DRIVER SHEET entries are standard for any driver. Refer to the Studio *Technical Reference Manual* for information about the configuration of the standard fields. The fields that require specific syntax for this driver are described below:



	Tag Name	Station	I/O Address	Action	Scan	Div	Add
	Filter text	Filter text	Filter text	(All)	(All)	Filter text	Filter text
1	CPU_Bits	2	MW0	Read+Write	Always		
2	T_130	2	DB2.DBD20	Read+Write	Always		
3	PT_131	2	DB5.DBF100	Read+Write	Always		

- **Station** field: Use this field to specify the PLC address (*PLC ID*). Valid values are 1– 255.
- **I/O Address:** Address of each register from the PLC. The syntax used in this field is described below:
  - For Flags, Timers, Counters, Inputs, and Outputs:

    <*Type*>[*Signed / Unsigned*]<*Format*><*Address*>.<*Bit*> (for example: **MSW1**) or

    <*Type*>[Signed/Unsigned]<*Format*><*Address*>.<*Bit*> (for example: **MB10.2, QX4.0**)

    <*Type*><*Format*><*Address*>.<*Len*> (for example: **MST2.10**) – String format only

  - For Data-Blocks:

    <*Type*><*TypeGroup*>[Signed/Unsigned]<*Format*><*Address*>.<*Bit*> (for example: **DB2B1**)

or **<*Type*><*TypeGroup*>.DB[Signed/Unsigned]<*Format*><*Address*>.<*Bit*>** (for example: **DB2.DBB1, DB2.DBSW10, DB3.DBD50, DB3.DBF30**)

<*Type*><*TypeGroup*><*Format*><*Address*>.<*Len*> (for example: **DB2ST2.10**) - String format only

Where:

- **[*Signed / Unsigned*]** (*optional parameter used for integer values only*): If you do not specify this parameter, Studio inserts an integer value based on the parameters you set in the *Communication Parameters* dialog. Valid values are **S** (signed) and **U** (unsigned).. Dword does not use **U** (Unsigned).

- **Type** is the register type. (**M**=Flags, **T**=Timers, **Z** or **C**=Counters, **E** or **I**=Inputs, **A** or **Q**=Outputs, and **DB**=Data Blocks, **ST**=String)

- **TypeGroup** is the group number of the configured register type (for Data-Block types only).

- **Address** is the device address. This number always refers to the **Byte** address number.

- **Format** defines how Studio treats the value read from or written to the device (**X**=Bit**,** **B**=Byte or Bit, **W**=Word, **D** or **DW**=Dword, **F**=Float, **ST**=String).
  - For Data-Blocks**:**
    T =Timer Format (S5T), **TS**=Timer in String format (hh:mm:ss:ms), **C**=Counter Format

- **Bit** is the bit number (from 0 – 7) from the **Byte** address. This parameter is optional, and it is supported only when the format Byte or Bit (**B** or **X**).

- ***<Len>*** is the length of characters to read or to write. It is in bytes. String format only.

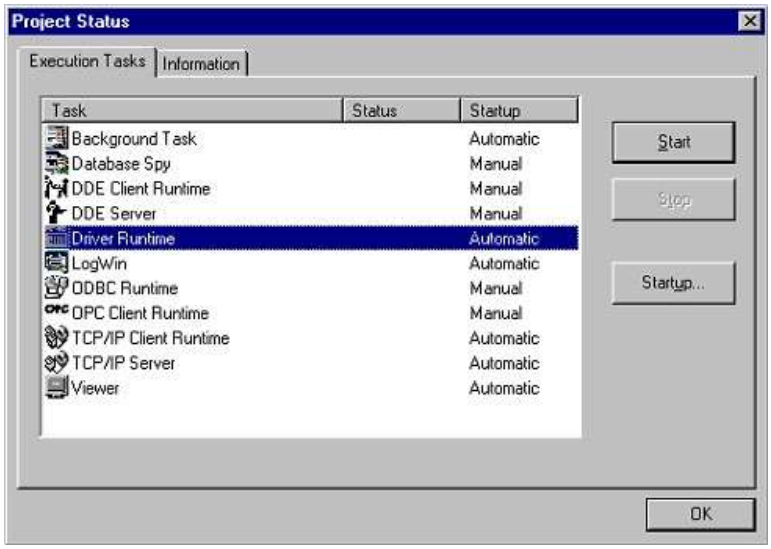
> **Note:**
> T, TS and C data formats are most used for Data-Blocks operand, however they can be used for all operands.

Executing the Driver

After adding the SIEME driver to a project, Studio sets the project to execute the driver automatically when you start the run-time environment.

To verify that the driver run-time task is enabled and will start correctly, perform the following steps:

1. Select **Project** → **Status** from the main menu bar.
   The *Project Status* dialog displays:



Project Status Dialog

2. Verify that the *Driver Runtime* task is set to **Automatic**.
   - If the setting is correct, click **OK** to close the dialog.
   - If the **Driver Runtime** task is set to **Manual**, select the **Driver Runtime** line. When the **Startup** button becomes active, click the button to toggle the *Startup* mode to **Automatic**.
3. Click **OK** to close the *Project Status* dialog.
4. Start the application to run the driver.

Troubleshooting

If the SIEME driver fails to communicate with the device, the tag you configured for the **Read Status** or **Write Status** fields will receive an error code. Use this error code and the following table to identify the failure that occurred.

Error Code	Description
-33	Invalid INI file
-34	Invalid Address
-36	Invalid Data Type
-38	Invalid Station
-39	Invalid Block Size
0	OK
1	Error to initialize driver
2	Maximum number of connections was exceeded.
244	Function is not supported
245	Internal error, please check
246	Listed job number is invalid
247	At least one parameter for opening the device is invalid
248	No free space in the request queue
249	The necessary class cannot be initialized
250	The necessary memory cannot be allocated
251	Device is not open
252	Device was not found
253	Device is already in use or open, or update is not valid
254	Function is not valid
255	End of program request
256	A parameter was not in the defined range
508	Not all parameters can be changed because the adapter is already initialized
509	The program "**AGLink_Config.EXE**" cannot be started
510	The parameters were changed because of plausibility checks
511	Parameter length is not supported
512	Requested option is not available
752	Sending buffer too small for packet
753	Receiving buffer too small for packet
754	Timeout while waiting for DLE after sending the packet
755	Packet to be sent is not correct (length 0 or NULL pointer)
756	After STX, arbitrary information was sent instead of DLE

Error Code	Description
757	After STX, NAK was sent instead of DLE
758	Timeout after initialization conflict (both have high priority)
759	Timeout while waiting for DLE after sending STX
760	Initialization conflict
761	Wrong protocol status
762	Checksum error
763	Timeout while waiting for checksum
764	Information after DLE was not DLE or ETX
765	Timeout while waiting for packet information (ZVZ)
766	Timeout while waiting for beginning of packet (QVZ)
767	Wrong information received instead of STX
768	Timeout while waiting for STX at the beginning of the program
1019	Adapter is not initialized
1020	Unknown error message from adapter
1021	Wrong MPI baud rate
1022	The address code is higher than HSA
1023	Requested adapter address already exists
1024	Received packet has wrong content
1260	Type (of data) is not supported
1261	Access to object is not permitted
1262	Invalid address
1263	Context is not supported
1264	PLC sends no data
1265	Function protection level is not sufficient
1266	Context is not supported
1267	Information cannot be determined at the moment
1268	Unknown error message from PLC; please check
1269	Wrong size operands or selected range too large
1270	Wrong operating status of PLC
1271	Error while restarting the PLC
1272	Error while starting the PLC
1273	Wrong PLC operating status
1274	Internal error; please check
1275	No data available (for example, missing DB)
1276	Hardware error (for example, nonexistent peripheral equipment)
1277	Number of frame does not fit
1278	PLC was not found

Error Code	Description
1279	No additional connection possible
1280	No connection to the requested PLC
1523	DSR signal changed to 0 (modem disconnected)
1524	DCD signal changed to 0 (no carrier)
1525	No connection to remote terminal
1526	No modem found at the device
1527	Error during initialization of auto-answer
1528	Error during initialization of dial tone
1529	Error during initialization of selection procedure
1530	Error during initialization sequence 4
1531	Error during initialization sequence 3
1532	Error during initialization sequence 2
1533	Error during initialization sequence 1
1534	Error during basis initialization (AT&FE0V1)
1535	Modem cannot hang up
1536	General modem error
1786	CIF card not logged in on the logical ring (bus)
1787	A resource error exists
1788	Wrong firmware version of CIF card
1789	Wrong hardware version of CIF card
1790	Error in a device driver function
1791	Requested board not found
1792	Requested device driver not found
2038	Close received instead of ReadOK
2039	Timeout while reading IP
2040	Error while reading IP
2041	Close received instead of WriteOK
2042	Timeout while writing IP
2043	Error while writing IP
2044	Close received instead of ConnectOK (for example, wrong rack or slot number)
2045	Timeout while establishing IP connection
2046	Error while establishing IP connection
2047	Listed IP address invalid
2048	Socket cannot be opened

⇨ **Tip:**

You can verify communication status using the Studio development environment *Output* window (*LogWin* module). To establish an event log for **Field Read Commands**, **Field Write Commands** and **Protocol Analyzer** right-click in the *Output* window. When the pop-up menu displays, select the option to set the log events.

If you are unable to establish communication with the PLC, try to establish communication between the PLC Programming Tool and the PLC. Quite frequently, communication is not possible because you have a hardware or cable problem, or a PLC configuration error. After successfully establishing communication between the device's Programming Tool and the PLC, you can retest the supervisory driver.

To test communication with Studio, we recommend using the sample application provided rather than your new application.

If you must contact us for technical support, please have the following information available:

- **Operating system and Project information**: To find this information, select **Help** → **Support Information**.
- **Driver version** and **communication log**: Displays in the Studio *Output* window when the driver is running.
- **Device model** and **boards**: Consult the hardware manufacturer's documentation for this information.

Sample Application

A sample application is provided in the **/COMMUNICATION EXAMPLES/SIEME** directory.
We strongly recommend that you use this sample application to test the SIEME driver before configuring your own customized application, for the following reasons:

- To better understand the information provided in this document.
- To verify that your configuration is working satisfactorily.
- To certify that the hardware used in the test (device, adapter, cable and PC) is working satisfactorily before you start configuring your own, customized applications.

> **Note:**
> This application sample is not available for all drivers.

Use the following procedure to perform the test:

1. Configure the device's communication parameters using the manufacturer's documentation.
2. Open and execute the sample application.
3. Execute the *Viewer* module in Studio to display information about the driver communication.

> **Tip:**
> You can use the sample application screen as the maintenance screen for your custom applications.

Revision History

Doc. Revision	Driver Version	Author	Date	Description of Changes
A	1.00	Eric Vigiani	14 Mar 03	First version
B	1.01	Eric Vigiani	01 Apr 03	Modified driver algorithm to avoid performance problems after running the application for a few minutes
C	2.01	Eric Vigiani	15 Sep 03	Modified the driver to resume communication when the cable is re-connected to the PLC
D	2.02	Leandro Coeli	27 Apr 05	Implemented String Type for Data Blocks
E	2.03	Leandro Coeli	26 Sep 05	Fixed problems with bit reading
F	2.04	Lourenço Teodoro	28 Mar 06	Fixed problems in function GetBit
G	2.05	Eric Vigiani	27 Jun 06	Implemented the Signed and Unsigned options.
H	3.01	Rafael R. Fernandes	26 Feb 08	Implemented String format Implemented "Initial ID Connection" field in Communication Parameters (to use more than one driver) Data types X and D (equivalent to Bit and DWord) were created. Driver was changed for Colon (:) to be optional in MAIN DRIVER SHEET New syntaxes to Data Blocks Operand were added. Included MPI Baud Rate option in Comm Parameters Included TS data type (Timer in string format).
I	10.1	Marcelo Carvalho	07 Jan 09	Updated driver version, no changes in the contents.
J	10.3	Marcelo Carvalho	30 Jul 09	Modified the driver to support SST format Fixed problem when connecting to multiple devices
K	10.4	Lourenço Teodoro	02 Dec 09	- Modified the driver to create less communication groups and optimize the communication
L	10.5	Fellipe Peternella	01 Apr 10	Fixed problem when communicating with multiple stations
M	10.6	André Körbes	21 Jul 10	- Siemens drivers are allowed to coexist on the same application - Fixed virtual group separation

N	10.7	André Körbes	1 Jul 2011	- Fixes on block size check
P	10.8	Ajay Anumalla	12 Mar 2013	- Updated driver version, no changes in the contents.
Q	10.8	Andre Bastos	17 Apr 2015	- Updated documentation only, no changes in the driver